

并行计算笔记整理(下)(gpt5.5 plus)

主讲：陈国良教授 原课程链接：[超算习堂EasyHPC 并行计算 - 结构·算法·编程](#)

P14

1. 第十四章总体：分布存储系统并行编程
2. 14.1 基于消息传递的编程
3. MPI 和 PVM 简介
4. 消息传递方式
5. 14.2 MPI 并行编程
6. MPI 中的消息
7. MPI 数据类型
8. 消息缓冲
9. MPI 中的消息信封
10. 通信子 communicator
11. MPI_Recv 和状态信息
12. MPI 中的四种通信模式
13. 点对点通信
14. MPI 群集通信
15. MPI 扩展
16. MPI 计算 π 的例子
17. 14.3 PVM 并行编程
18. 并行虚拟机的构造
19. PVM 中的进程管理
20. PVM 中的通信
21. PVM 计算 π 的例子
22. 14.4 基于数据并行的并行编程
23. 14.5 HPF 并行编程
24. HPF 简介
25. HPF 的基本特性
26. HPF 的 FORALL 机制
27. HPF 数据映射
28. 高斯消去法的 HPF 程序
29. 本章三种编程方式对比
30. 本章核心总结
31. 复习重点

P15

1. 第十五章总体：并行程序设计环境与工具
2. 15.1 软件工具与环境
3. 15.2 并行编译器
4. 15.3 并行程序调试
5. 并行程序调试步骤
6. 并行程序调试技术
7. 15.4 并行程序性能分析
8. 并行程序性能预测
9. 并行程序性能监控
10. 静态分析和动态分析比较
11. 性能分析方法
12. 并行程序性能可视化
13. 科学数据的特征化
14. 可视化技术
15. 15.5 图形化并行程序集成开发环境
16. GRADE 图形应用开发环境
17. 本章核心总结
18. 复习重点

P14

1. 第十四章总体：分布存储系统并行编程

这份 **PC14.pdf** 讲的是 **第十四章：分布存储系统并行编程**。它和上一章共享存储编程正好相对：共享存储中线程可以直接访问共享变量，而分布存储中每个进程有自己的地址空间，进程之间必须通过 **消息传递** 或 **数据并行语言机制** 交换信息。

1.1 本章结构

课件第 2 页列出五节：

- 14.1 基于消息传递的编程
- 14.2 MPI 并行编程
- 14.3 PVM 并行编程
- 14.4 基于数据并行的并行编程
- 14.5 HPF 并行编程

可以简单理解为：

```
分布存储系统并行编程
├── 显式消息传递
│   ├── MPI
│   └── PVM
└── 数据并行编程
    └── HPF
```

2. 14.1 基于消息传递的编程

2.1 基本思想

在消息传递模型中，一个并行应用由一组进程组成。

每个进程：

```
只能直接访问自己的私有数据
不能直接访问其他进程地址空间
通过 send / receive 传递消息
通过消息交换实现数据共享和同步
```

这和共享内存模型不同。共享内存中线程可以直接读写共享变量；消息传递中进程必须显式发送和接收数据。

2.2 消息传递库

课件列出了一些流行的消息传递软件，例如：

名称	创建者	特点
CMMD	Thinking Machines	使用 Active Messages 降低延迟
Express	Parasoft	支持群集通信和 I/O
Fortran-M	Argonne National Lab	支持模块化和确定性
MPI	MPI Forum	广泛采用的标准
NX	Intel	起源于 Intel hypercube MPP
P4	Argonne National Lab	结合共享内存和消息传递
PVM	Oak Ridge National Lab	广泛使用的独立系统
UNIFY	Mississippi State	允许 MPI 和 PVM 调用
Zipcode	Livermore National Lab	贡献了 context 概念

课件建议开发新的消息传递应用时使用 **MPI 或 PVM**，因为它们容易获得，并且多数厂家支持。

3. MPI 和 PVM 简介

3.1 MPI 简介

MPI 是 **Message Passing Interface**，消息传递接口。

课件给出的 MPI 发展过程包括：

```
1992年4月：组建消息传递接口标准工作组
1992年10月：初稿形成，主要定义点对点通信接口
1993年1月：第一届 MPI 会议在 Dallas 举行
1994年5月：MPI 标准正式发布
1997年：MPI-2 发布，原来的 MPI 更名为 MPI-1
```

MPI 的目标是提供一个：

实际可用
可移植
高效
灵活

的消息传递接口标准。

MPI 是语言无关的接口库，并提供 C、Fortran 和 Java 等语言绑定。

3.2 MPI 的实现

MPI 可以建立在厂家专用环境中，例如：

IBM SP2 的 POE/MPL
Intel Paragon 的 OSF/Nx

也可以使用公共 MPI 环境，例如：

CHIMP
LAN
MPICH

课件特别提到，MPICH 是 MPI 在多种机器上的可移植实现，可以安装在：

PC
工作站
SMP
MPP
COW

3.3 PVM 简介

PVM 是 **Parallel Virtual Machine**，并行虚拟机。

课件给出的信息是：

开发时间：始于1989年
开发单位：Tennessee大学、Oak Ridge国家实验室、Emory大学
特点：适应性好、可扩展、可移植、易使用、源码可免费获得

PVM 可以把多台异构计算机组织成一个“虚拟并行机”，让用户像使用一台并行机一样运行程序。

3.4 MPI 和 PVM 的主要差别

课件列出三点：

1. PVM 是自包含系统，MPI 不是。
2. MPI 对消息传递提供更强大的支持。
3. PVM 不是标准，因此版本更新更灵活。

第一点最关键。

PVM 自己包含进程管理和 I/O 等功能；MPI 依赖底层平台提供进程管理和 I/O 能力。

不过两者后来功能逐渐趋同：MPI-2 增加了进程管理功能，PVM 也提供了更多群集通信函数。

4. 消息传递方式

4.1 通信模式需要理解的三个问题

课件第 11 页指出，理解通信模式时要关注：

共有多少个进程？
进程间如何同步？
如何管理通信缓冲区？

消息传递系统常用三种通信模式：

同步消息传递
阻塞消息传递
非阻塞消息传递

4.2 同步消息传递

同步通信要求发送方和接收方都到达对应的发送和接收点。

发送返回意味着：

消息已经被收到

优点是语义明确，不需要临时系统缓冲区。

缺点是等待时间最大，通信和计算不能重叠。

4.3 阻塞消息传递

阻塞发送的开始条件是发送方到达发送点。

发送返回意味着：

消息已经发送完

接收返回意味着：

消息已经收到

它介于同步和非阻塞之间，可能需要系统提供临时缓冲区。

4.4 非阻塞消息传递

非阻塞发送返回只意味着：

已经通知系统某个消息要被发送

非阻塞接收返回只意味着：

已经通知系统某个消息要被接收

它可以让通信与计算重叠，但需要额外的状态检查，例如 `wait` 或 `test`。

4.5 非阻塞通信的额外开销

课件第 15 页指出，非阻塞模式也会带来额外开销：

临时缓冲区内存空间
分配缓冲区操作
消息拷入和拷出临时缓冲区
额外的检测和等待函数

所以非阻塞通信不是“免费加速”。它只有在计算和通信能真正重叠时才有意义。

4.6 消息传递模型的优缺点

优点：

用户可以完全控制并行性开发、数据分布和通信实现

缺点：

通信位置、数据移动、数据复制、一致性都要程序员显式处理
科学计算中显式域分解很困难
不容易通过逐步修改串行程序实现并行化

这一点和 OpenMP 形成对比。OpenMP 可以在原串行代码中逐步插入并行指令，而 MPI 程序通常需要从数据分布开始整体重构。

5. 14.2 MPI 并行编程

5.1 本节结构

课件第 18 页列出 MPI 并行编程的七个主题：

- 1 MPI 中的消息
- 2 MPI 中的消息信封
- 3 MPI 中的四种通信模式
- 4 点对点通信
- 5 群集通信
- 6 MPI 扩展
- 7 例子：计算Pi的MPI程序

6. MPI 中的消息

6.1 MPI 程序基本结构

课件给出一个计算：

$\sum \text{foo}(i)$

的 MPI SPMD 程序。

核心函数包括：

```
MPI_Init(&argc, &argv);
MPI_Comm_size(MPI_COMM_WORLD, &group_size);
MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);
MPI_Send(...);
MPI_Recv(...);
MPI_Finalize();
```

含义分别是：

函数	作用
MPI_Init	初始化 MPI 环境
MPI_Comm_size	获取进程组大小
MPI_Comm_rank	获取当前进程编号
MPI_Send	发送消息
MPI_Recv	接收消息
MPI_Finalize	结束 MPI 环境

6.2 MPI 进程的特点

课件强调：

MPI进程是重型的单线程进程。
它们拥有不同的地址空间。
一个进程不能直接访问另一个进程地址空间中的变量。

因此，进程间通信必须靠消息传递实现。

6.3 MPI 消息的组成

课件把 MPI 消息分为两部分：

消息缓冲 message buffer
消息信封 message envelope

消息缓冲是“信的内容”，即真正要发送的数据。

消息信封是“信封上的地址信息”，说明消息发给谁、用什么标签、属于哪个通信子。

6.4 MPI_Send 的参数

典型发送调用：

```
MPI_Send(&N, 1, MPI_INT, i, i, MPI_COMM_WORLD)
```

一般形式：

```
MPI_Send(buffer, count, datatype, destination, tag, communicator)
```

其中：

buffer, count, datatype 定义消息缓冲
destination, tag, communicator 定义消息信封

也就是说，MPI 不只是发送一个地址，还必须说明数据项数量、数据类型、目的进程、标签和通信范围。

7. MPI 数据类型

7.1 为什么需要消息数据类型

课件给出两个原因：

支持异构计算
允许发送非连续、非均匀内存区中的消息

异构计算中，不同机器可能使用不同的数据表示方式。MPI 数据类型能帮助系统在不同平台之间正确解释数据。

7.2 MPI 预定义数据类型

课件列出一些常见预定义类型：

MPI 类型	C 类型
MPI_CHAR	signed char
MPI_DOUBLE	double
MPI_FLOAT	float

MPI 类型	C 类型
MPI_INT	int
MPI_LONG	long
MPI_SHORT	short
MPI_UNSIGNED	unsigned int
MPI_PACKED	打包数据

Fortran 中也有对应类型，例如：

```
MPI_INTEGER
MPI_REAL
MPI_DOUBLE_PRECISION
MPI_CHARACTER
MPI_LOGICAL
```

7.3 非连续数据和导出数据类型

如果要发送数组中的偶数编号元素：

```
A[0], A[2], A[4], ...
```

这些元素在内存中不是连续存放的。普通的：

```
(address, count, datatype)
```

无法直接描述这种消息。

MPI 提供 **derived datatype**，即导出数据类型。

例如课件用：

```
MPI_Type_vector(50, 1, 2, MPI_DOUBLE, &EvenElements);
MPI_Type_commit(&EvenElements);
MPI_Send(A, 1, EvenElements, destination, ...);
```

表示从数组 `A` 中每隔一个元素取一个双精度数，共取 50 个。

7.4 MPI_Type_vector 的含义

形式是：

```
MPI_Type_vector(count, blocklength, stride, oldtype, &newtype)
```

含义：

```
count: 块的数量
blocklength: 每块中连续 oldtype 数据项数量
stride: 相邻块之间相隔多少个 oldtype 元素
oldtype: 已有类型
newtype: 构造出的新类型
```

例如：

```
MPI_Type_vector(50, 1, 2, MPI_DOUBLE, &EvenElements)
```

表示：

50 个块
每块 1 个 double
相邻块间隔 2 个 double

因此正好取出偶数位置元素。

8. 消息缓冲

8.1 用户消息缓冲

第一种缓冲是用户程序中声明的变量或数组。

例如：

```
send(A, 16, Q, tag)
```

这里的 `A` 就是用户缓冲。

8.2 系统消息缓冲

第二种缓冲由消息传递系统创建和管理。

它用于发送消息时暂存数据，用户程序中不直接出现。

8.3 用户级临时缓冲

第三种是 MPI 允许用户划出一块内存，作为任意消息的中间缓冲。

课件图示中比较了：

只使用用户缓冲
使用系统缓冲 `S`
使用用户级临时缓冲 `T`

9. MPI 中的消息信封

9.1 消息信封三要素

MPI 发送例程中的信封由三项组成：

```
destination  
tag  
communicator
```

其中：

destination: 目的进程编号
tag: 消息标签
communicator: 通信子

9.2 为什么需要 tag

课件给了一个例子。

如果进程 P 连续发送两个消息：

```
send(A, 32, Q)  
send(B, 16, Q)
```

进程 Q 连续接收：


```
recv(X, 32, P)
recv(Y, 16, P)
```

如果没有标签，后发送的 `B` 可能先到达，就可能被第一个 `recv` 错误接收到 `X` 中。

使用标签后：

```
send(A, 32, Q, tag1)
send(B, 16, Q, tag2)

recv(X, 32, P, tag1)
recv(Y, 16, P, tag2)
```

就能避免消息类型混淆。

9.3 tag 与服务请求

另一个例子中，两个客户进程 `P` 和 `R` 向服务进程 `Q` 发送请求。

如果使用不同标签：

```
tag1 表示一种服务请求
tag2 表示另一种服务请求
```

服务进程可以通过 `Status.Tag` 判断消息类型，从而按不同方式处理。

10. 通信子 communicator

10.1 通信子的定义

课件定义：

```
通信子 = 进程组 + 上下文
```

进程组是有限有序进程集合，大小为 `group_size`，进程编号为：

```
0, 1, ..., n-1
```

常用查询函数：

```
MPI_Comm_size(communicator, &group_size);
MPI_Comm_rank(communicator, &my_rank);
```

10.2 为什么需要通信子

仅靠 `tag` 不够安全。

因为不同库函数或用户代码可能不小心使用相同 `tag`。

通信子通过不同上下文隔离通信，即使两个通信子包含相同进程组，也不会混淆消息。

这对并行库开发特别重要。

10.3 MPI_Comm_dup

```
MPI_Comm_dup(MPI_COMM_WORLD, &myworld)
```

创建一个新的通信子 `myworld`。

它包含和 `MPI_COMM_WORLD` 相同的进程组，但有不同上下文。

10.4 MPI_Comm_split

课件给出 10 个进程的例子：

```
Color = my_rank % 3;
Key = my_rank / 3;
MPI_Comm_split(Myworld, Color, Key, &Splitworld);
```

含义是：

按 Color 把原通信子拆成多个子通信子
按 Key 决定新通信子中的 rank 顺序

这可以把全体进程划分成多个通信组。

11. MPI_Recv 和状态信息

11.1 MPI_Recv 参数

形式是：

```
MPI_Recv(address, count, datatype, source, tag, communicator, status)
```

其中：

source 和 tag 可以使用通配符
MPI_ANY_SOURCE
MPI_ANY_TAG

11.2 Status 的作用

接收完成后，MPI_Status 中保存：

Status.MPI_SOURCE：实际源进程编号
Status.MPI_TAG：实际消息标签

实际收到的数据项数可以通过：

```
MPI_Get_count(&Status, MPI_INT, &C)
```

得到。

11.3 Status 适用场景

当一个服务进程可能从不同源进程接收不同大小、不同标签的消息时，Status 很有用。

典型模式是：

```
MPI_Recv(..., MPI_ANY_SOURCE, MPI_ANY_TAG, comm, &Status);

switch (Status.MPI_TAG) {
    case tag_0: ...
    case tag_1: ...
}
```

12. MPI 中的四种通信模式

12.1 同步发送 synchronous

同步发送要求相应接收已经启动后发送才返回。

它不需要接收端额外缓冲，但等待较强。

MPI 对应函数：

```
MPI_Ssend
MPI_Issend
```

12.2 缓冲发送 buffered

缓冲发送要求用户先提供缓冲区：

```
MPI_Buffer_attach(buffer, size)
```

结束时可以：

```
MPI_Buffer_detach(...)
```

对应函数：

```
MPI_Bsend
MPI_Ibsend
```

12.3 标准发送 standard

标准发送可能是同步的，也可能是缓冲的，取决于 MPI 实现。

对应函数：

```
MPI_Send
MPI_Isend
```

12.4 就绪发送 ready

就绪发送要求用户确信相应接收已经开始。

它可以使用更高效协议，但如果接收没有开始就调用，就可能出错。

对应函数：

```
MPI_Rsend
MPI_Irsend
```

13. 点对点通信

13.1 MPI 点对点函数表

课件列出：

类型	阻塞	非阻塞
Standard Send	MPI_Send	MPI_Isend
Synchronous Send	MPI_Ssend	MPI_Issend
Buffered Send	MPI_Bsend	MPI_Ibsend
Ready Send	MPI_Rsend	MPI_Irsend
Receive	MPI_Recv	MPI_Irecv

类型	阻塞	非阻塞
Completion Check	MPI_Wait	MPI_Test

13.2 进程流水线例子

课件给出三进程流水线：

```
W → P → X → Q → Y → R → Z
```

其中进程 Q 逻辑是：

```
接收 X
计算 Y = Q(X)
发送 Y
```

为了重叠通信和计算，可以使用非阻塞接收和非阻塞发送：

```
MPI_Irecv(...);
MPI_Isend(...);
CurrentY = Q(CurrentX);
```

13.3 双缓冲

为了避免正在发送的数据被修改，课件给出双缓冲思想：

```
Xbuf0 / Xbuf1
Ybuf0 / Ybuf1
```

一组缓冲用于当前计算，另一组缓冲用于通信。
通过 `MPI_Wait` 确认通信完成后再复用缓冲。

13.4 非阻塞通信中的错误例子

课件给出一个可能死锁或结果不确定的例子。

问题在于：

```
MPI_Issend 是非阻塞同步发送
MPI_Irecv 可能由于条件不满足而没有启动
```

可能结果包括：

```
死锁
打印 Y=0
打印 Y=5
```

这说明非阻塞通信虽然灵活，但必须认真处理完成检查和数据依赖。

14. MPI 群集通信

14.1 Broadcast 广播

形式：

```
MPI_Bcast(Address, Count, Datatype, Root, Comm)
```

Root 进程把相同消息发送给通信子中的所有进程。

14.2 Scatter 播撒

形式：

```
MPI_Scatter(SendAddress, SendCount, SendDatatype,
            RecvAddress, RecvCount, RecvDatatype,
            Root, Comm)
```

Root 给每个进程发送不同的数据块。

14.3 Gather 聚集

形式：

```
MPI_Gather(SendAddress, SendCount, SendDatatype,
            RecvAddress, RecvCount, RecvDatatype,
            Root, Comm)
```

Root 从每个进程收集一块数据，并按 rank 顺序放入接收缓冲。

14.4 Allgather 全局收集

形式：

```
MPI_Allgather(...)
```

每个进程都收集到所有进程的数据。

它可以理解为：

每个进程都执行一次 `gather` 的效果

14.5 Alltoall 全局交换

形式：

```
MPI_Alltoall(...)
```

每个进程都向所有进程发送一个消息，同时也从所有进程接收一个消息。

课件指出，一次 all-to-all 中共有：

n^2 个消息通信

这是通信量很大的模式。

14.6 Reduction 归约

形式：

```
MPI_Reduce(SendAddress, RecvAddress, Count, Datatype,
            Op, Root, Comm)
```

所有进程贡献局部值，通过 `Op` 归约成一个结果，存入 Root。

常见操作：

求和
求最大值
求最小值
乘积

14.7 Scan 扫描

形式：

```
MPI_Scan(SendAddress, RecvAddress, Count, Datatype, Op, Comm)
```

scan 不需要 Root，因为它会为每个进程生成一个前缀结果。

例如 rank 0 到 rank k 的值按某个操作组合后，结果存入 rank k。

14.8 Barrier 路障

形式：

```
MPI_Barrier(Comm)
```

通信子中的所有进程互相等待，直到所有进程都调用了 `MPI_Barrier` 才能继续。

14.9 群集通信共同特点

课件总结了群集通信的几个关键规则：

通信子中的所有进程必须调用群集通信例程
`Count` 和 `Datatype` 在相关进程中必须一致
群集例程中没有 `tag` 参数
`MPI-1` 只支持阻塞的通信子内群集通信

如果只有部分进程调用群集通信，而其他进程没有调用，行为是不确定的，可能死锁，也可能得到错误结果。

15. MPI 扩展

15.1 MPI-2 对 MPI-1 的扩展

课件列出 MPI-2 的扩展包括：

动态进程
单边通信
非阻塞群集通信
通信子间群集通信
`MPI-IO`
`Fortran90` 和 `C++` 绑定
实时处理支持
面向 `profiling`、`monitoring`、`debugging` 的外部接口

15.2 动态进程

MPI-1 不定义如何创建进程，必须依赖平台。

MPI-2 支持动态进程，优点包括：

可移植地创建进程
有利于 `PVM` 程序移植到 `MPI`
支持 `client/server` 和 `process farm`
更有效利用资源和负载平衡
支持一定容错能力

15.3 MPI_Spawn

课件给出形式：

```
MPI_Spawn(command_line, minprocs, maxprocs, info,
          root, comm, intercomm, array_of_errcodes)
```

它尝试派生子进程，并通过 `intercomm` 建立父进程组和子进程组之间的通信。

15.4 单边通信 RMA

MPI-2 引入远程存储访问：

```
RMA = Remote Memory Access
```

单边通信允许一个进程把数据放到另一个进程，或从另一个进程取数据，而不需要另一边显式参与对应的 send/receive。

相关函数：

```
MPI_Get
MPI_Put
```

课件强调，MPI-1 点对点通信都是双边通信，发送者和接收者都必须参与。

16. MPI 计算 π 的例子

16.1 计算思想

仍然使用积分公式：

$$\pi = \int_0^1 \frac{4}{(1+x^2)} dx$$

MPI 程序中：

```
每个进程计算一部分区间
每个进程得到局部 mypi
通过 MPI_Reduce 求总和 pi
```

16.2 关键 MPI 操作

程序中使用：

```
MPI_Bcast(&n, 1, MPI_LONG, 0, MPI_COMM_WORLD);
```

把区间数 `n` 从 0 号进程广播给所有进程。

然后：

```
MPI_Reduce(&mypi, &pi, 1, MPI_DOUBLE,
          MPI_SUM, 0, MPI_COMM_WORLD);
```

把所有进程的局部积分结果求和到 0 号进程。

16.3 编译和运行

课件给出：

```
mpicc -o pi pi.c
mpirun -np 2 pi
```

其中 `-np 2` 表示启动 2 个 MPI 进程。

17. 14.3 PVM 并行编程

17.1 本节结构

课件第 75 页列出：

- 1 并行虚拟机的构造
- 2 PVM中的进程管理
- 3 PVM中的通信
- 4 例子：计算Pi的PVM程序

18. 并行虚拟机的构造

18.1 PVM 系统组成

PVM 系统由两部分组成：

```
PVM监控进程 pvmd
用户可调用库 libpvm3.a
```

`pvmd` 常驻在虚拟机的每台主机上。

`libpvm3.a` 与用户程序链接，用于进程管理、消息传递和虚拟机管理。

18.2 PVM 中的术语

课件说明：

```
node 在 PVM 中称为 host
process 在 PVM 中称为 task
```

并行虚拟机包括：

```
1到多台主机
1个 master pvmd
0到多个 slave pvmd
```

18.3 PVM 控制台

安装 PVM 后，可以输入：

```
pvm host_file
```

启动 PVM 控制台。

成功后出现：

```
pvm>
```

表示当前处于 PVM 控制台模式。

18.4 PVM 控制命令

课件列出常用命令：

命令	含义
add apple	把主机 apple 加入虚拟机
delete apple	从虚拟机删除主机 apple
conf	列出虚拟机配置
spawn -count 4 app	启动 4 个任务运行 app
jobs	列出运行作业
halt	中止所有 PVM 进程并关闭 PVM

18.5 host_file

PVM 的硬件配置在主机表文件 `host_file` 中说明。

每台主机有一个 host descriptor，保存主机配置、packet queues 和通信缓冲等信息。

当新 slave 加入虚拟机后，master 会更新主机表，并把修改后的主机表广播到整个虚拟机，使所有主机保持一致。

18.6 动态配置虚拟机

PVM 支持动态加入和删除主机。

相关函数：

```
pvm_addhosts(...)
pvm_delhosts(...)
```

例如：

```
info = pvm_addhosts(hosts, nhost, infos);
info = pvm_delhosts(hosts, nhost, infos);
```

课件第 81 页图展示了动态加入主机 H3 的过程：用户任务请求加入主机，master pvmd 与 slave pvmd 通信，更新主机表并提交配置。

19. PVM 中的进程管理

19.1 静态并行程序

PVM 支持静态并行。

例如：

```
pvm> spawn -count 4 foo
```

会创建 4 个任务执行同一个 SPMD 程序 `foo`。

如果 `foo` 不再调用动态创建函数，那么程序的并发度就是静态的 4。

19.2 程序到主机的映射

PVM 支持两种映射：

缺省映射：由 PVM 选择主机
用户自定义映射：用户指定主机或体系结构

例如：

```
pvm> spawn -(apple) foo
```

表示在主机 apple 上启动任务。

```
pvm> spawn -(RS6K) foo
```

表示在任意 RS/6000 节点上启动任务。

19.3 动态进程管理

PVM 支持 MPMD 和动态进程管理，关键函数是：

```
pvm_spawn(...)
```

其参数包括：

```
prog: 可执行文件名
argv: 参数数组
flag: 选项
where: 与 flag 配合指定位置
ntask: 启动副本数
tid: 保存派生任务 ID
```

19.4 pvm_spawn 的 flag

课件列出：

flag	含义
PvmTaskDefault	主机由 PVM 选择
PvmTaskHost	主机由用户定义
PvmTaskArch	体系结构由用户定义
PvmTaskDebug	调试状态启动
PvmTaskTrace	生成跟踪数据
PvmMppFront	在 MPP 前端启动
PvmHostCompl	使用 complement 主机集合

19.5 PVM 进程管理函数

常见函数包括：

```
pvm_mytid
pvm_parent
pvm_exit
pvm_spawn
pvm_kill
pvm_pstat
pvm_tasks
pvm_mstat
pvm_config
```

其中：

pvm_mytid: 获取当前任务 ID
pvm_parent: 获取父任务 ID
pvm_exit: 退出 PVM
pvm_spawn: 派生任务
pvm_kill: 终止任务
pvm_config: 获取虚拟机配置信息

19.6 PVM 动态分组

PVM 支持动态分组。

任何任务可以在任意时候加入或离开一个组。

PVM 分组由 group server daemon 管理，第一次调用组函数时自动启动。

灵活性很高，但会带来非确定性：

任务加入或离开组可能改变广播结果
成员离开组可能导致 barrier 死锁

19.7 PVM 组函数

课件列出：

函数	含义
pvm_joingroup	加入组并获得成员编号
pvm_lvgroup	离开组
pvm_gettid	根据组内编号获得任务 ID
pvm_getinst	根据任务 ID 获得组内编号
pvm_gsize	获取组大小
pvm_barrier	组内路障同步
pvm_bcast	向组广播消息
pvm_reduce	归约操作

20. PVM 中的通信

20.1 三类通信

课件第 90 页给出 PVM 通信类型：

pvmd 间通信
pvmd 与任务间通信
两个任务间通信

图中显示，任务间通信可以经过 TCP，pvmd 间通信也可能通过 UDP 等协议辅助实现。

20.2 PVM 通信函数

常见函数包括：

```
pvm_initsend(...)
pvm_pkint(...)
pvm_send(...)
pvm_mcast(...)
pvm_recv(...)
pvm_nrecv(...)
pvm_trecv(...)
pvm_probe(...)
pvm_buinfo(...)
pvm_upkint(...)
```

典型流程是：

```
初始化发送缓冲
打包数据
发送消息
接收消息
解包数据
```

21. PVM 计算 π 的例子

21.1 程序思想

PVM 版本也把区间 `[0,1]` 分成 N 个区域，并启用 n 个任务。

每个任务负责：

```
N/n 个区域
```

计算局部和 `mypi`。

最后通过：

```
pvm_reduce(...)
```

把各任务局部结果归约成总和。

21.2 关键步骤

初始任务：

```
加入组 PI
派生 n-1 个子任务
读入 N
把 N 多播给子任务
```

子任务：

```
接收 N
参与 barrier
计算局部积分
参与 reduce
```

最后：

```
0号任务输出结果
所有任务离开组并退出 PVM
```

21.3 编译和运行

课件给出：

```
pvmcc -o pi pi.c
pvmd -d7 host_file &
pvm
pvm> spawn -4 pi
pvm> halt
```

22. 14.4 基于数据并行的并行编程

22.1 数据并行模型特点

课件第 100 页列出数据并行模型特点：

- 单线程
- 在密集数据结构上的并行操作
- 全局命名空间
- 隐式同步和通信
- 隐式计算和数据分布
- 可移植性好

它适用于：

- SIMD
- MIMD
- 共享存储器
- 分布存储器

22.2 数据并行模型的目标

数据并行模型希望在分布存储器上实现全局命名空间，使程序员不必显式写通信和数据分布。

也就是说，程序员写类似数组整体操作，编译器负责把数据分配到处理器并生成通信。

22.3 与消息传递的比较

消息传递：

控制力强，但程序员负担大

数据并行：

减轻程序员负担，但依赖好的数据分布

课件指出，数据并行编程的成败在很大程度上取决于程序员能否确定好的数据分布。

22.4 数据并行语言的发展

课件列出：

- IVTRAN
- Kali
- Fortran D
- Vienna Fortran
- Fortran 90 / Fortran 95
- C* / CM Fortran / *Lisp
- HPF / HPF+ / F-- / HPC++ / HPJava

其中 HPF 是本章后面重点讲的高性能 Fortran。

23. 14.5 HPF 并行编程

23.1 本节结构

课件第 103 页列出：

- 1 高性能 Fortran (HPF) 简介
- 2 HPF 的数据并行机制
- 3 例子：高斯消去法的 HPF 程序

24. HPF 简介

24.1 HPF 发展历史

课件给出：

1991年：Supercomputing'91 上提出高性能 Fortran 版本
1992年1月：第一次 HPF 会议
1992年3月：HPF 论坛成立
1993年5月：发布 HPF 1.0，基于 Fortran90
1994年11月：发布 HPF 1.1
1997年1月：发布 HPF 2.0，基于 Fortran95

HPF 的目标是统一分布存储机器上的 Fortran 并行编程方法。

24.2 对 HPF 的批评

课件列出三种批评：

有人认为 HPF 太高层，不如 MPI 实用
有人认为 HPF 太低层，未来编译技术和体系结构可避开这些指令
有人认为抽象层次合适，但仍需扩充才能适应未来应用

也就是说，HPF 处在“高层抽象”和“显式控制”之间，存在争议。

24.3 HPF 的设计目标

课件列出三个目标：

支持数据并行程序设计
发挥非均匀存储访问的 MIMD 和 SIMD 计算机性能
能够调整代码适应各种体系结构

24.4 HPF 2.0 的组成

HPF 2.0 包括：

语言基本部分
已核准扩展部分
已承认的外部接口

基本部分是任何 HPF 编译器必须支持的特性。

扩展部分满足高级需要。

外部接口用于提供额外服务。

25. HPF 的基本特性

25.1 数据并行说明

用于表达可并行性：

```
INDEPENDENT
REDUCTION
```

其中：

```
INDEPENDENT: 声明循环无依赖，可并行执行
REDUCTION: 标识归约变量
```

25.2 数据映射说明

用于控制数据分布：

```
ALIGN
DISTRIBUTE
```

它们用于声明数据如何分布到处理器上，以减少通信并保持负载均衡。

25.3 新的内部函数和库函数

包括：

```
归约函数
组合分散函数
前缀/后缀函数
排序函数
位操作函数
```

这些函数帮助利用特定机器的低层能力。

25.4 外部过程

HPF 支持 extrinsic procedures，用于和其他编程方法结合，支持混合语言编程。

26. HPF 的 FORALL 机制

26.1 简单 FORALL

例子：

```
FORALL (K=1, M-1) X(K+1)=X(K)
```

含义是对所有满足条件的 `K` 同时执行赋值。

另一个例子：

```
FORALL (I=1:N, J=1:N) X(I,J)=Y(J,I)
```

这表示把 `Y` 的转置赋给 `X`。

26.2 带限制的 FORALL

例子：

```
FORALL (i=2:5, x(i)>0) x(i)=x(i-1)+x(i+1)
```

这里只有满足 $x(i)>0$ 的活动索引才执行赋值。

课件示例中，如果初始：

```
x = {1, -1, 2, -2, 3, -3}
```

活动索引是：

```
{3, 5}
```

因此只更新 $x(3)$ 和 $x(5)$ 。

26.3 多个 for-三元组

例子：

```
FORALL(i=1:2, j=1:3, y(i,j)>0)  
  z(i,j)=1/y(i,j)
```

这里索引是二维联合索引：

```
(i,j)
```

有效集合为：

```
{(1,1),(1,2),(1,3),(2,1),(2,2),(2,3)}
```

再从中选出满足 $y(i,j)>0$ 的活动集合。

26.4 FORALL 结构

FORALL 结构可以包含多条语句：

```
FORALL (I=2:9)  
  A(I) = A(I-1) + A(I+1)  
  B(I) = A(I)  
ENDFORALL
```

课件强调执行语义：

```
先对所有 I 求右端表达式  
再整体赋给 A  
然后再把新 A 的值赋给 B
```

这体现了 FORALL 的并行赋值语义。

27. HPF 数据映射

27.1 数据映射的定义

数据映射就是：

把数据分布到处理器上

在分布存储系统上，必须先对数组划分，再分配到处理器，计算才能在各处理器上进行。

数据映射目标：

最小化通信开销
负载均匀分布

27.2 Owner-compute 规则

课件提到 HPF 编译器可以使用 owner-compute 规则：

数据在哪个处理器上，就在哪个处理器上执行相关计算

因此数据分布会直接决定计算负载分布。

27.3 PROCESSORS

`PROCESSORS` 用于描述虚拟处理器结构。

例子：

```
!HPF$ PROCESSORS P4(4)
```

表示 4 个虚拟处理器的一维结构。

```
!HPF$ PROCESSORS P22(2,2)
```

表示 2×2 的二维虚拟处理器结构。

27.4 ALIGN

`ALIGN` 用于描述数据对准。

例子：

```
!HPF$ ALIGN A(I) WITH B(I)
```

表示 `A(I)` 和 `B(I)` 分配到同一个处理器。

```
!HPF$ ALIGN A(I,J) WITH B(J,I)
```

表示 `A(I,J)` 和 `B(J,I)` 对准，适合转置关系。

27.5 不同维数组对准

例子：

```
!HPF$ ALIGN A(I,*) WITH B(I)
```

表示 `A` 的第 `I` 行所有元素与 `B(I)` 对准。

```
!HPF$ ALIGN A(I) WITH B(I,*)
```

表示 $A(I)$ 被重复对准到 B 第 I 行各元素所在处理器。

27.6 DISTRIBUTE

`DISTRIBUTE` 用于描述数据分布。

主要方式：

BLOCK
CYCLIC

27.7 BLOCK 分布

例子：

```
!HPF$ DISTRIBUTE A(BLOCK) ONTO P4
```

把数组 A 分成 4 个连续块，分配给 4 个处理器。

```
!HPF$ DISTRIBUTE A(BLOCK(30)) ONTO P4
```

表示每块 30 个元素，最后一块可能不足 30。

27.8 二维 BLOCK 分布

例子：

```
!HPF$ DISTRIBUTE B(BLOCK, BLOCK) ONTO P22
```

表示二维数组 B 在行、列两个方向都分块，分到 2×2 处理器结构上。

```
!HPF$ DISTRIBUTE B(*, BLOCK) ONTO P4
```

表示行方向不分布，只在列方向分块。

27.9 CYCLIC 分布

例子：

```
!HPF$ DISTRIBUTE A(CYCLIC) ONTO P4
```

表示从 $A(1)$ 开始，把元素循环分配给 4 个处理器。

```
!HPF$ DISTRIBUTE A(CYCLIC(2)) ONTO P4
```

表示每两个元素一组，循环分配。

例如：

处理器1: $A(1), A(2), A(9), A(10), \dots$
处理器2: $A(3), A(4), A(11), A(12), \dots$

27.10 数据映射例子

课件给出代码：

```
integer A(100), B(100), C(101), i
!HPF$ ALIGN A(i) WITH B(i-1)
!HPF$ PROCESSOR N(4)
!HPF$ DISTRIBUTE A(BLOCK) ONTO N
!HPF$ DISTRIBUTE C(CYCLIC) ONTO N
FORALL (i=2:100)
    A(i) = A(i)+B(i-1)
    C(i) = C(i-1)+C(i)+C(i+1)
ENDFORALL
```

含义是：

A 和 B 根据下标偏移进行对准
A 用 BLOCK 分布
C 用 CYCLIC 分布
FORALL 表达并行数组计算

课件第 121 页图把这个过程分成：

ALIGN: 逻辑对准
DISTRIBUTE: 映射到虚拟处理器
Physical Mapping: 映射到实际机器

28. 高斯消去法的 HPF 程序

28.1 程序结构

课件最后给出一个 HPF 高斯消去法程序。

关键声明：

```
parameter n=32
real A(n,n+1), x(n)
!HPF$ PROCESSOR Nodes(4)
!HPF$ ALIGN x(i) WITH A(i,j)
!HPF$ DISTRIBUTE A(BLOCK,*) ONTO Nodes
```

含义是：

定义4个虚拟处理器
x(i) 与 A(i,j) 对准
A 的第一维按 BLOCK 分布
A 的第二维不分布

28.2 消元阶段

核心循环：

```
do i=1,n-1
    pivot_location = MAXLOC(ABS(A(i:n,i)))
    swap(...)
    A(i,i:n+1)=A(i,i:n+1)/A(i,i)
    FORALL (j=i+1:n,k=i+1:n+1)
        A(j,k)=A(j,k)-A(j,i)*A(i,k)
    end do
```

其中：

MAXLOC 用于选主元
swap 交换主元行
FORALL 并行更新剩余矩阵区域

这体现了 HPF 在稠密线性代数中的数据并行写法。

28.3 回代阶段

回代代码：

```
do i=n,1,-1
  x(i)=A(i,n+1)
  A(1:i-1,n+1)=A(1:i-1,n+1)-A(1:i-1,i)*x(i)
end do
```

每求出一个 $x(i)$ ，就更新前面方程的右端项。

28.4 数据分布

课件最后说明，当 $n=32$ 且有 4 个节点时：

结点1: A(1:8, 1:33), x(1:8)
结点2: A(9:16, 1:33), x(9:16)
结点3: A(17:24, 1:33), x(17:24)
结点4: A(25:32, 1:33), x(25:32)

这说明 A 的行被分块分给四个节点。

29. 本章三种编程方式对比

方式	代表	核心思想	优点	缺点
消息传递	MPI	显式 send/receive	控制力强、性能好、适合分布存储	程序员负担大
虚拟机消息传递	PVM	把异构主机组成虚拟并行机	动态灵活、自包含系统	非标准，行为可能非确定
数据并行	HPF	全局数组操作 + 数据分布指令	编程抽象高、减轻通信负担	依赖编译器和良好数据分布

30. 本章核心总结

第十四章的主线是：

分布存储系统中，每个进程只能访问本地数据：
要实现并行计算，必须解决数据如何分布、消息如何传递、进程如何同步的问题。

30.1 MPI 的核心

MPI = 标准化、可移植、高效的消息传递接口

重点掌握：

```
MPI_Init / Finalize
MPI_Comm_size / rank
MPI_Send / Recv
datatype
tag
communicator
Status
Bcast / Scatter / Gather / Alltoall / Reduce / Scan / Barrier
```

30.2 PVM 的核心

PVM = 把多台主机组织成并行虚拟机

重点掌握：

```
pvmd
host_file
pvm console
pvm_spawn
pvm_send / recv
pvm_group
pvm_reduce
```

30.3 HPF 的核心

HPF = 用 Fortran 风格的数据并行机制描述分布存储计算

重点掌握：

```
FORALL
PROCESSORS
ALIGN
DISTRIBUTE
BLOCK
CYCLIC
owner-compute
```

31. 复习重点

这一章最应该掌握：

1. 消息传递模型中进程只能访问私有数据
2. MPI 和 PVM 是最重要的消息传递系统
3. 同步、阻塞、非阻塞三种消息传递方式的区别
4. 非阻塞通信可以重叠计算，但需要 `wait/test` 检查
5. MPI 消息 = 消息缓冲 + 消息信封
6. MPI_Send 六个参数的含义
7. tag 用于区分消息类型，communicator 用于隔离通信上下文
8. MPI derived datatype 用于描述非连续或混合类型消息
9. MPI 四种发送模式：standard、synchronous、buffered、ready
10. MPI 点对点通信和群集通信函数
11. MPI-2 的动态进程、RMA、MPI-IO
12. PVM 的 pvmd、host、task、动态主机配置和动态分组
13. 数据并行模型的特点：全局命名空间、隐式通信、显式数据分布
14. HPF 的 FORALL、ALIGN、DISTRIBUTE、BLOCK、CYCLIC
15. HPF 高斯消去法中数据按行块分布，FORALL 用于并行更新

一句话概括：

第十四章讲的是分布存储系统怎样写并行程序：MPI 和 PVM 代表显式消息传递，程序员直接控制通信；HPF 代表数据并行，让程序员通过数组操作和数据分布指令表达并行性，由编译器生成通信。

P15

1. 第十五章总体：并行程序设计环境与工具

这份 PC15.pdf 讲的是 **第十五章：并行程序设计环境与工具**。前面几章已经讲了并行程序设计模型、共享存储编程、分布存储编程；这一章则转向“写并行程序时需要哪些工具支持”，包括编译器、调试工具、性能分析工具和图形化集成开发环境。

1.1 本章结构

课件第 2 页列出五节：

- 15.1 软件工具与环境

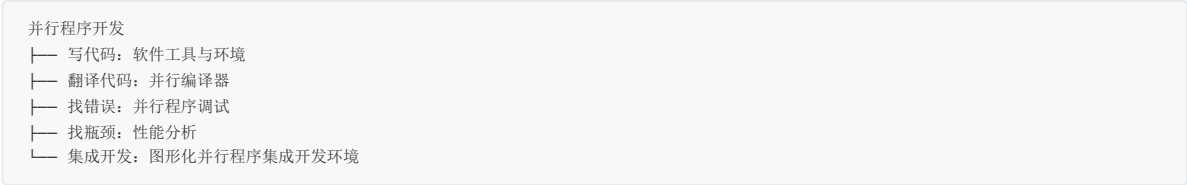
15.2 并行编译器

15.3 并行程序调试

15.4 并行程序性能分析

15.5 图形化并行程序集成开发环境

这一章的主线可以概括为：



2. 15.1 软件工具与环境

2.1 软件工程与环境的基本内容

课件第 3 页把软件工具和环境分成四类：

- 编码工具

软件工程工具

集成工具

将来的工具与环境

这说明并行程序设计环境不是单一工具，而是一整套支持程序开发生命周期的工具系统。

2.2 编码工具

编码工具主要服务于“写程序、翻译程序、定位语法错误”。

课件列出：

- 编辑器

编译器

连接器和加载器

预处理程序

交叉引用程序

源级查错器

查错辅助程序

可以这样理解：

工具	作用
编辑器	写源代码
编译器	把源代码翻译成目标代码
连接器	把多个目标文件和库连接成可执行文件
加载器	把程序装入内存执行
预处理程序	在编译前处理宏、头文件、条件编译等
交叉引用程序	分析变量、函数、模块之间的引用关系
源级查错器	在源代码级别调试程序
查错辅助程序	辅助定位语法、运行或逻辑错误

对于并行程序来说，源级调试和交叉引用尤其重要，因为多个进程或线程之间的数据关系更复杂。

2.3 软件工程工具

软件工程工具主要支持大型程序开发和维护。

课件列出：

系统构造程序
版本管理程序
设计编辑器
代码产生器
测试辅助程序

这些工具解决的问题不是“单次运行程序”，而是：

如何组织大型程序？
如何管理多个版本？
如何自动生成部分代码？
如何做测试？
如何多人协同开发？

并行程序通常规模较大，涉及不同模块、不同处理器、不同通信库，因此软件工程工具非常重要。

2.4 集成工具

课件列出三类集成：

数据集成
公共前端
控制集成

2.4.1 数据集成

数据集成指不同工具之间可以共享数据。

例如编译器产生的符号表、调试器使用的源码信息、性能分析器生成的运行轨迹，都应该能被其他工具读取。

2.4.2 公共前端

公共前端指多个工具使用统一界面。

例如用户可以在同一个环境里完成：

编辑
编译
运行
调试
性能分析

而不是频繁切换多个独立工具。

2.4.3 控制集成

控制集成指一个工具可以调用和控制另一个工具。

例如 IDE 中点击“调试”，系统自动完成编译、连接、加载、启动调试器等步骤。

2.5 将来的工具与环境

课件列出未来工具方向：

过程工具
群件工具
可视化工具
程序分析工具

这些方向说明未来并行程序设计环境会更重视：

开发过程管理
多人协同
图形化表达
自动程序分析

并行程序越复杂，越需要工具帮助程序员理解程序行为。

3. 15.2 并行编译器

3.1 并行编译器的作用

并行编译器的任务不是简单把代码翻译成机器指令，而是要识别、转换和优化程序中的并行性。

课件第 5 页列出并行编译器相关内容：

编译及其并行化
相关分析
代码优化
代码生成

3.2 编译及其并行化

课件把并行化分为两类：

向量化 SIMDizing
并行化 MIMDizing

3.2.1 向量化 SIMDizing

向量化是把普通循环转换成适合 SIMD 或向量机执行的形式。

例如：

```
for (i = 0; i < n; i++)  
    C[i] = A[i] + B[i];
```

可以向量化为一次处理多个元素。

它适合：

数组运算
矩阵运算
规则循环
数据并行程序

3.2.2 并行化 MIMDizing

并行化是把程序转换成适合多个处理器或多个进程并行执行的形式。

例如把循环迭代分给多个处理器：

```
P0 处理一部分循环  
P1 处理一部分循环  
...
```

它更适合 MIMD 系统，如多核处理器、共享内存系统、分布式内存系统等。

3.3 相关分析

并行编译器判断一段代码能不能并行化，关键要做 **相关分析**。

课件列出四种相关：

流相关
反相关
输出相关
控制相关

3.4 流相关

流相关也叫真相关。

形式是：

先写后读

例如：

```
A[i] = ...  
x = A[i];
```

后面的读依赖前面的写。

如果两个语句之间存在流相关，不能随意交换顺序，否则可能读到错误数据。

3.5 反相关

反相关是：

先读后写

例如：

```
x = A[i];  
A[i] = ...
```

如果把写提前，前面的读就会读到新值而不是旧值。

反相关本质上来自变量复用，有时可以通过重命名变量消除。

3.6 输出相关

输出相关是：

先写后写

例如：

```
A[i] = 1;  
A[i] = 2;
```

两个写操作都写同一个位置，顺序不同会影响最终结果。

如果多个并行任务同时写同一变量，就可能出现输出相关或写冲突。

3.7 控制相关

控制相关来自条件分支。

例如：

```
if (x > 0)  
    A[i] = 1;
```

这条赋值是否执行依赖于条件 `x > 0`。

控制相关会影响并行化，因为编译器需要判断不同分支之间是否可以并行执行。

3.8 代码优化

课件列出两类优化：

代码向量化方法
代码并行化方法

3.8.1 代码向量化方法

目标是让程序适合向量处理器或 SIMD 指令。

例如：

循环展开
循环交换
内存连续访问
消除循环相关

3.8.2 代码并行化方法

目标是把程序拆成多个可并行任务。

例如：

循环并行化
任务划分
数据划分
同步插入
通信生成
负载均衡优化

3.9 代码生成

课件说代码生成是：

将中间形式的代码转换成可执行的具体机器目标代码

也就是说，并行编译器前面先把源程序分析成中间表示，再做相关分析和优化，最后生成具体机器可以执行的代码。

对于并行系统，代码生成可能还要生成：

线程创建代码
消息传递代码
同步代码
数据分布代码
通信调度代码

所以并行编译比普通串行编译复杂得多。

4. 15.3 并行程序调试

4.1 并行程序调试为什么难

课件第 7 页指出并行程序调试的困难主要有两个：

不确定性
探针效应

4.2 不确定性

并行程序的执行顺序可能每次都不同。

例如两个线程同时访问共享变量：

线程 A 先执行，结果一种；
线程 B 先执行，结果另一种。

或者 MPI 程序中消息到达顺序不同，也可能导致程序行为不同。

这意味着：

同一个 bug 可能不是每次运行都出现

这比串行程序调试难得多。

4.3 探针效应

探针效应是指：

调试工具本身会改变程序行为。

例如加入打印语句、断点或监控代码后，程序执行速度和进程调度顺序发生变化。

结果可能是：

不加调试代码时程序出错；
加了调试代码后错误消失。

这在并发程序中非常常见。

4.4 并行程序调试方法

课件列出两种方法：

重放 Replay
断点调试 Breakpoint Debugging

4.4.1 重放 Replay

重放是记录程序运行中的关键事件，之后按相同顺序重新执行程序。

对并行程序来说，关键是记录：

消息发送和接收顺序
同步事件顺序
线程调度事件
共享变量访问顺序

这样可以让一次偶然出现的错误被重复观察。

4.4.2 断点调试 Breakpoint Debugging

断点调试是在程序某些位置暂停执行，检查变量和状态。

并行程序中的断点比串行程序复杂，因为可能需要：

暂停一个进程
暂停一组进程
暂停所有进程
在某个全局条件满足时暂停

5. 并行程序调试步骤

课件第 7 页给出五个步骤。

5.1 第一步：先确保串行程序正确

如果程序有串行版本，应该先保证串行版本正确。

因为如果串行算法本身就有错误，直接调并行程序会更困难。

5.2 第二步：单机执行并行程序

先在单机上运行并行程序，确保基本逻辑正确。

例如：

```
用 1 个处理器运行 MPI 程序
用 1 个线程运行 OpenMP 程序
```

这样可以排除一部分并行调度和通信问题。

5.3 第三步：逐步增大处理器数

在单处理器正确后，再逐步增加处理器数：

```
1 → 2 → 4 → 8 → ...
```

这样可以观察错误从什么时候开始出现。

如果 1 个处理器正确，2 个处理器错误，说明问题很可能出在：

```
数据划分
通信
同步
共享变量访问
```

5.4 第四步：逐步增加并行成分

不要一开始就把所有部分都并行化。

可以先并行化一个循环或一个模块，确认正确后，再继续增加并行部分。

这样便于定位错误。

5.5 第五步：出错时检查关键机制

课件指出出错时要依次检查：

```
数据定义
数据分布
同步机制
```

对于分布式程序，还要检查：

```
消息是否发送给正确进程
tag 是否匹配
通信子是否正确
数据类型和长度是否一致
```

对于共享内存程序，还要检查：

```
变量是 private 还是 shared
是否需要锁
是否缺少 barrier
是否存在数据竞争
```

6. 并行程序调试技术

6.1 课件列出的调试技术

课件第 8 页列出：

全局断点
渐增检查点
事件分析
静态分析

6.2 全局断点

全局断点是让多个进程或线程在某个全局条件下同时停止。

例如：

所有进程到达某一通信点时暂停
某个变量在任意进程中满足条件时暂停

这适合检查分布式程序整体状态。

6.3 渐增检查点

检查点是保存程序当前状态。

渐增检查点不是每次保存完整状态，而是只保存相对于上次检查点的变化。

这样可以减少存储开销。

它常用于：

错误恢复
程序重放
长时间并行计算调试

6.4 事件分析

课件说事件分析法记录运行中事件轨迹信息，事后加以分析。

事件包括：

进程启动
消息发送
消息接收
锁申请
锁释放
barrier 到达
barrier 退出

轨迹信息主要用于：

阅览
重放
定位死锁
分析通信顺序

6.5 静态分析

静态分析是不运行程序，直接分析源代码或中间代码。

它可以发现：

潜在数据竞争
未初始化变量
可能死锁的通信模式
数组越界
不匹配的 `send / receive`

静态分析的优点是开销小，但缺点是可能不够精确。

6.6 断点调试的三种形式

课件列出：

控制流断点
自陷断点
谓词断点

6.6.1 控制流断点

在程序某一行或某一语句处暂停。

6.6.2 自陷断点

程序运行到某个特殊指令或陷阱时暂停。

6.6.3 谓词断点

当某个条件表达式为真时暂停。

例如：

```
rank == 3 && x < 0
```

这比普通断点更适合并行调试，因为错误常常只发生在某个进程或某种条件下。

6.7 并行程序的性能调试

课件第 8 页还提到性能调试包括：

测量
分析
优化

其中分析又包括：

静态分析
动态分析

这说明并行程序调试不只是找“错”，还要找“慢”。

7. 15.4 并行程序性能分析

7.1 性能分析的主要内容

课件第 10 页把性能分析分成三部分：

性能预测
性能监控
性能可视化

可以理解为：

运行前预测
运行中或运行后测量
用图形帮助理解

8. 并行程序性能预测

8.1 静态分析就是性能预测

课件说：

并行程序性能的静态分析又叫性能预测

它是在程序运行前，根据程序结构和机器模型估计性能。

8.2 性能预测方法

课件列出：

分析预测 Analytical Prediction
模拟仿真

8.2.1 分析预测

分析预测通常用数学模型估计运行时间。

例如：

$T = \text{计算时间} + \text{通信时间} + \text{同步时间}$

或者：

$T_p = w/p + \text{通信开销} + \text{同步开销}$

它适合做理论分析，但可能忽略真实系统细节。

8.2.2 模拟仿真

模拟仿真需要建立两个模型：

并行系统建模
应用程序建模

并行系统模型描述机器，例如处理器数、网络拓扑、带宽、延迟。

应用程序模型描述程序，例如计算量、通信模式、同步频率。

8.3 性能评估参数选择

性能分析需要选择合适参数。

常见参数包括：

运行时间
加速比
效率
可扩展性
负载平衡
通信时间
同步等待时间
I/O时间
缓存命中率

如果只看总运行时间，很难知道程序慢在哪里。

9. 并行程序性能监控

9.1 动态分析就是性能监控

课件说：

并行程序性能的动态分析又叫性能监控

动态分析是在程序实际运行时收集性能数据。

9.2 性能监控工具

课件列出两类工具：

联机的
脱机的

9.2.1 联机工具

联机工具在程序运行时实时显示性能状态。

例如显示：

当前负载
通信量
CPU利用率
内存使用

9.2.2 脱机工具

脱机工具先记录运行轨迹，程序结束后再分析。

它适合复杂程序，因为运行时显示所有信息可能干扰程序执行。

9.3 监控方式

课件列出：

时钟驱动监控
事件驱动监控

9.3.1 时钟驱动监控

按固定时间间隔采样。

例如每 10 毫秒记录一次每个进程状态。

优点是实现简单。

缺点是可能漏掉短时间事件。

9.3.2 事件驱动监控

当特定事件发生时记录。

例如：

发送消息时记录
进入 `barrier` 时记录
获得锁时记录

优点是能准确捕捉关键事件。

缺点是如果事件太多，记录开销可能很大。

10. 静态分析和动态分析比较

10.1 静态分析

课件第 11 页说，静态分析采用模拟或分析方法，在源程序一级获取性能参数报告。

优点：

时间代价较小
可以针对重要程序结构做多种性能选择

缺点：

准确性较差

因为它没有真实运行，很多硬件行为、调度行为、缓存行为难以准确预测。

10.2 动态分析

动态分析采用测量方法收集程序运行中的性能参数。

优点：

数据较准确

缺点：

灵活性较差

因为它只能分析已经运行过的情况，改变问题规模、处理器数或算法结构后，需要重新运行。

11. 性能分析方法

课件第 11 页列出四个分析方法。

11.1 统计程序各部分执行时间

先找出程序时间花在哪里：

初始化
计算
通信
同步
I/O
结果输出

这一步是性能分析的基础。

11.2 分析大计算量程序段的瓶颈

对于耗时大的程序段，要判断它主要瓶颈是：

计算部分
通信部分
I/O部分

例如一个并行矩阵乘法，如果大部分时间在等待数据传输，那优化计算代码意义不大，应该优化通信。

11.3 比较理论加速和实测加速

根据理论值和实测值分析：

计算粒度是否太小
负载是否平衡
通信开销是否过大
存储访问是否冲突
CACHE 命中率是否低

例如理论上应有 8 倍加速，实际只有 3 倍，就要检查通信、同步和负载平衡。

11.4 根据效率和可扩放性分析结构与算法组合

并行程序不只是当前处理器数下快，还要能扩展。

需要看：

处理器数增加后效率下降快不快
问题规模增加后能否保持性能
算法结构是否适合目标机器

这就是可扩展性分析。

12. 并行程序性能可视化

12.1 性能可视化的三个环节

课件第 10 页列出：

数据生成
数据显示
数据分析与用户交互

含义是：

先收集性能数据
再用图形方式展示
最后让用户交互式分析瓶颈

12.2 可视化定义

课件第 12 页定义科学数据可视化：

使用图形方法增强对科学数据的解释，并便于用户直观理解。

也就是说，可视化不是简单画图，而是帮助人理解复杂数据。

12.3 可视化目的

课件说可视化的目的是：

为科学数据提供富有表达性的、有效可视的表示概念、方法和工具

对于并行程序来说，可视化可以帮助理解：

哪个进程忙？
哪个进程空闲？
哪里通信多？
哪里等待时间长？
是否负载不均？
是否出现通信热点？

12.4 可视化基本过程

课件给出基本过程：

真实现象 → 数字 → 图画 → 目标表示

在并行性能分析中可以对应为：

程序运行行为
→ 性能数据
→ 时间线图、通信图、负载图
→ 程序员理解瓶颈

13. 科学数据的特征化

13.1 可视元素 Cues

课件列出可视元素：

空间位置和运动
长度
深度
面
体
厚度
角度
斜率取向
密度
颜色
对比度

这些元素可以用来表达性能信息。

例如：

颜色表示进程状态
长度表示执行时间
线条粗细表示通信量
空间位置表示处理器拓扑

13.2 定量表示

课件列出不同数据的表示方式：

点数据集：散布图和图示符
连续函数：标量表示
矢量：长度和方向表示
n维张量场：主方向和绝对值表示

这部分虽然来自科学可视化，但同样可以用于性能数据表示。

14. 可视化技术

课件第 13 页列出多种可视化技术。

14.1 散布图 Scatter plots

点数据用：

(x_i, y_i)

表示。

例如可以用横轴表示时间，纵轴表示处理器编号，点表示某个事件发生。

14.2 图示符 Glyphs

图示符用于表示复杂集合。

例如一个小图标可以同时表示：

CPU利用率
通信量
等待时间
内存占用

14.3 直方图和饼图

直方图用矩形面积表示数值大小。

例如显示每个进程的计算时间。

饼图表示部分与总体的比例。

例如：

计算时间占 60%
通信时间占 25%
同步等待占 10%
I/O 占 5%

14.4 等值线和等值面

等值线用于二维数据集，线上的点具有相同值。

等值面用于三维数据集，面上的点具有相同值。

它们常用于科学计算结果显示，也可用于并行程序中的空间性能分布分析。

14.5 图像显示

二维数据可以用不同数值、灰度和颜色显示。

例如热力图可以显示：

哪个处理器负载高
哪个时间段通信密集
哪个区域缓存命中率低

14.6 射线跟踪

射线跟踪用于三维体数据可视化。

课件说它把三维体元素 voxel 投影到二维上。

这常用于三维科学数据，例如流体、医学图像、体渲染。

14.7 动画

动画通过连续移动画面产生动感。

对于并行程序分析，动画可以显示：

任务迁移过程
消息流动过程
负载变化过程
通信热点形成过程

15. 15.5 图形化并行程序集成开发环境

15.1 基本思想

课件第 15 页讲图形化并行程序开发方法。

基本思想是：

用节点表示计算
用弧表示计算之间的交互
采用统一图形用户界面
把设计、编辑、编译连接、调试和性能分析集成起来
实现并行程序开发各阶段的可视化

也就是说，程序员不只是写文本代码，而是可以通过图形方式表达并行程序结构。

15.2 节点和弧的含义

在图形化并行程序中：

节点 = 计算任务
弧 = 任务之间的通信或依赖

例如一个流水线程序可以画成：

P1 → P2 → P3

一个分治程序可以画成树状结构。

这种图形表达有助于理解并行结构。

15.3 基本组成

课件说图形化并行程序集成环境至少包括：

可视化的程序设计工具
可视化的模拟系统
可视化的程序调试和行为分析工具

15.3.1 可视化程序设计工具

用于设计和构造并行程序。

15.3.2 可视化模拟系统

用于在真实运行前模拟程序行为。

15.3.3 可视化调试和行为分析工具

用于观察程序执行过程，分析通信、同步、负载等行为。

15.4 基本流程

课件第 15 页列出四步：

1. 用图形语言编辑器设计和构造并行程序
2. 用预编译器产生 C 源代码、MAKE 文件和交叉引用文件
3. 由生成器连接有关库函数生成可执行文件
4. 可执行代码以调试或跟踪模式加载到处理器上执行

这说明图形化环境并不是脱离传统编译流程，而是在图形层之上自动生成传统程序文件。

16. GRADE 图形应用开发环境

16.1 GRADE 是什么

课件第 16 页介绍：

GRADE = Graphical Application Development Environment

即图形应用开发环境。
它是一个图形化并行程序集成开发环境。

16.2 GRADE 中开发并行程序的过程

课件列出：

并行程序设计
映射
预编译
编译
调试
性能分析

这个过程覆盖了并行程序完整生命周期。

16.3 第 17 页图的整体含义

课件第 17 页给出 GRADE 工作流程图。

图中从上到下可以理解为：

GRAPNEL / GRED
↓
GRP
↓
DMS&S 映射
↓
GRP
↓
GRP2C 预编译
↓
C/C++ 源代码、Makefile、交叉引用文件
↓
C 编译器和连接器
↓
可执行文件
↓
调试 / 监视 / 性能分析 / 可视化

16.4 GRAPNEL 和 GRED

图中最上方是：

GRAPNEL
GRED

它们用于并行程序设计阶段。
可以理解为：

GRAPNEL：图形化并行程序语言或表示
GRED：图形编辑器

用户通过它们设计并行程序结构。

16.5 DMS&S 映射

图中映射阶段是：

DMS&S

它负责把图形化并行程序映射到实际处理器或运行平台。

也就是决定：

哪些任务放在哪些处理器上
任务之间通信如何实现

16.6 GRP2C 预编译

图中预编译阶段是：

GRP2C

它把图形化程序表示转换为：

C/C++ 源代码
Makefile
交叉引用文件

这说明图形化环境最终还是要生成普通源代码供编译器编译。

16.7 编译和连接

之后由：

C 编译器
连接器

生成可执行文件。

还会连接相关库，例如图中右侧的：

PVM 库
Tape/PVM 库
GRAPNEL 库

说明 GRADE 可以和 PVM 等并行运行库结合。

16.8 调试、监视和性能分析

图中下方包括：

DDBG
Tape/PVM
PROVE

可以理解为：

工具	作用
DDBG	调试
Tape/PVM	监视和跟踪
PROVE	可视化 and 性能分析

程序运行产生跟踪文件，然后通过可视化工具分析程序行为。

17. 本章核心总结

17.1 本章讲什么

第十五章讲的是并行程序设计的工具支撑体系：

并行程序设计环境与工具

- └─ 软件工具与环境
- └─ 并行编译器
- └─ 并行程序调试
- └─ 并行程序性能分析
- └─ 图形化并行程序集成开发环境

17.2 最重要的理解

并行程序开发比串行程序更需要工具支持，因为并行程序有：

多个执行单元
复杂数据分布
通信和同步
非确定性行为
性能瓶颈多样
调试困难

所以并行程序设计环境不仅要帮助写代码，还要帮助：

发现并行性
检查相关关系
调试并发错误
分析通信和同步瓶颈
可视化程序行为
集成开发流程

18. 复习重点

这一章最应该掌握：

1. 软件工具与环境包括编码工具、软件工程工具、集成工具和未来工具
2. 并行编译器要做向量化和并行化
3. 相关分析包括流相关、反相关、输出相关和控制相关
4. 并行程序调试难在不确定性和探针效应
5. 调试方法包括重放和断点调试
6. 调试步骤是先串行正确，再单机并行，再逐步增加处理器数
7. 调试技术包括全局断点、检查点、事件分析和静态分析
8. 性能分析包括性能预测、性能监控和性能可视化
9. 静态分析准确性较差但代价小，动态分析数据准确但灵活性较差
10. 性能分析要区分计算、通信、同步和 **I/O** 瓶颈
11. 可视化技术包括散点图、图符号、直方图、饼图、等值线、等值面、图像显示、射线跟踪和动画
12. 图形化并行开发环境用节点表示计算，用弧表示交互
13. **GRADE** 的流程包括设计、映射、预编译、编译、调试和性能分析

一句话概括：

第十五章讲的是并行程序开发不能只靠程序员手写和肉眼检查，而需要编译器、调试器、性能分析器和图形化集成环境共同支持；其中调试解决“程序对不对”，性能分析解决“程序快不快”，图形化环境则帮助程序员更直观地设计、运行和理解并行程序。